Computer Practical
Solar System Model

Planetary dynamics and Fourier analysis

Paul Connolly • September 2026

1 Overview

This practical develops the command-line modelling workflow from the previous exercise by applying it to a larger numerical model. You will compile and run a three-dimensional Solar System model written in Fortran, inspect its configuration, analyse its NetCDF output with Python, and use Fourier methods to identify periodic behaviour.

The emphasis is not on understanding every line of Fortran or Python. Concentrate on the governing equations, the small number of parameters in `namelist.in`, and what the model output tells you about orbital motion and interactions between bodies.

By the end of the practical you should be able to:

- relate Newton's law of gravitation to a coupled system of ordinary differential equations (ODEs);
- compile and run a Fortran model and modify a Fortran namelist;
- interpret orbital trajectories and Sun–planet distance time series;
- use a Fourier spectrum to identify orbital periods, harmonics and an interaction timescale;
- explain how mutual planetary interactions perturb ideal two-body orbits;
- explain the connection between stellar reflex motion ("wobble") and exoplanet detection.

2 From gravity to a system of ODEs

Let body i have position $\mathbf{r}_i = (x_i, y_i, z_i)$, velocity $\mathbf{v}_i$, and mass m_i. The displacement from body i to body j is

$$\mathbf{r}_{ij} = \mathbf{r}_j - \mathbf{r}_i, \tag{1}$$

with separation

$$r_{ij} = \left|\mathbf{r}_{ij}\right| = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2 + (z_j - z_i)^2}. \tag{2}$$

Newton's law of gravitation gives the acceleration of body i due to body j as

$$\mathbf{a}_{ij} = G m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{r_{ij}^3}. \tag{3}$$

The direction is automatically towards body j. Adding the contributions from all active gravity sources gives

$$\frac{d^2\mathbf{r}_i}{dt^2} = G \sum_{j \neq i} m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{|\mathbf{r}_j - \mathbf{r}_i|^3}. \tag{4}$$

As in the harmonic-oscillator example from the previous practical, a second-order equation is written as coupled first-order equations:

$$\frac{d\mathbf{r}_i}{dt} = \mathbf{v}_i, \qquad \frac{d\mathbf{v}_i}{dt} = G \sum_{j \neq i} m_j \frac{\mathbf{r}_j - \mathbf{r}_i}{|\mathbf{r}_j - \mathbf{r}_i|^3}. \tag{5}$$

For ten bodies in three dimensions this gives 60 first-order ODEs: three position components and three velocity components for each body.

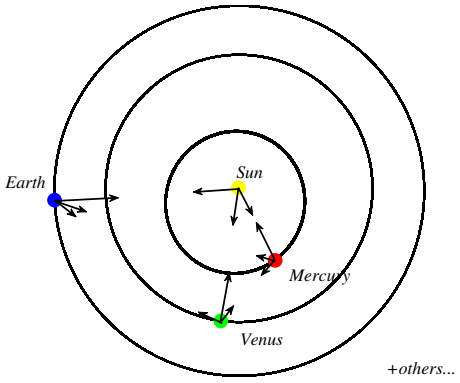

Figure 1: Schematic of the model. Each body can feel the gravitational acceleration produced by the bodies enabled as gravity sources. The sketch is schematic and not to scale.

The code stores the product Gm_j as the gravitational parameter `gm`. The values in `namelist.in` are supplied in $\mathrm{km}^3\ \mathrm{s}^{-2}$ and are converted internally to SI units before the integration.

A useful distinction from the previous practical: the Solar System model uses the adaptive ODE solver DVODE. The namelist variable `dt` is the requested output interval and also the maximum internal DVODE step; it is not a fixed Forward Euler time-step.

3 Log in, download and compile the code

Use the same two-window SSH/SFTP workflow as in *Tools of the Trade*. On campus, the teaching-server address is `130.88.55.57`; otherwise use the server address given for the course.

In your SSH window:

```
ssh -Y YOUR_USERNAME@SERVER_ADDRESS
```

Open a second terminal window for file transfer:

```
sftp YOUR_USERNAME@SERVER_ADDRESS
```

Your password will not be displayed while you type it.

In the SSH window, clone the repository and enter it:

```
git clone https://github.com/EnvModelling/solar-system-model
cd solar-system-model
pwd
ls
```

If you already have a copy from an earlier session, enter that directory instead of cloning it again and use `git pull` to update it.

Compile the Fortran model:

```
make
```

A successful build creates the executable `main.exe`. The teaching server already provides the required compiler and NetCDF libraries.

4 Inspect the model configuration

Open the namelist with line numbers displayed:

```
nano -l namelist.in
```

In nano, `Ctrl-W` searches for text. Search first for `&run_vars`, then for `interact`. Line numbers may change when the repository is updated, so searching for parameter names is more reliable than relying on a fixed line number.

Fortran uses an exclamation mark (`!`) to begin a comment. Logical values are written as `.true.` and `.false.`; scientific notation such as `1.d4` means 1×10^4 in double precision.

The ten entries of `interact(1:10)` correspond, in order, to the Sun, Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune and Pluto. An entry set to `.true.` means that body contributes to the Newtonian gravitational acceleration. In the supplied default, only the Sun is enabled as a gravity source, so the planets feel the Sun but do not perturb one another.

Table 1: Key run parameters in `namelist.in`.

Parameter	Default	Meaning
`interact(1:10)`	Sun only	Bodies enabled as Newtonian gravity sources.
`tfinal`	1.d4	Integration duration in Earth years.
`dt`	30.d0	Requested output interval in days and maximum DVODE internal step.
`interval_io`	10.	Interval in years between progress messages.
`general_relativity`	.false.	Turns the supplied approximate solar relativistic correction on or off.
`gm`	Solar-system values	Gm for each body; input units are $\mathrm{km}^3\ \mathrm{s}^{-2}$.

Table 2: Approximate orbital periods used as a guide in the exercises.

Planet	Period (yr)	Planet	Period (yr)
Mercury	0.25	Jupiter	11.9
Venus	0.60	Saturn	29.5
Earth	1.00	Uranus	84.0
Mars	1.90	Neptune	164.8
		Pluto	248.0

5 Run the default model

The supplied configuration is the *Sun-only* or *Keplerian* case. Run it with:

```
./run.sh
```

The helper script writes the trajectory to `/tmp/$USER/output.nc`. The run may take a little while. You can check the output from the SSH shell with:

```
ls -lh /tmp/$USER/
```

The practical uses the same small Python plotting scripts as the previous version of the exercise. They are deliberately simple: each one reads `/tmp/$USER/output.nc` and writes a named figure back to the same temporary directory.

First plot the planetary orbits:

```
python3 python/plot_solar_system01.py
```

This creates `/tmp/$USER/orbits.png`. In SFTP, download it with a descriptive local filename:

```
get /tmp/YOUR_USERNAME/orbits.png orbits_keplerian.png
```

Now plot planet–Sun distance against time:

```
python3 python/plot_solar_system02.py
```

This creates `/tmp/$USER/milankovitch.png`. Download it as:

```
get /tmp/YOUR_USERNAME/milankovitch.png milankovitch_keplerian.png
```

In SFTP, `$USER` is not expanded by the remote shell, so use your actual username. Giving each downloaded result a different local name is important because later runs will overwrite the files in `/tmp/$USER/`.

6 Experiments

Keep a practical record. Save each figure using a meaningful filename and make brief notes as you go. Record what was changed between runs and what you infer from each plot. You may be asked to interpret these experiments in the assessment.

6.1 Keplerian orbits, perihelion and aphelion

With only the Sun enabled as a gravity source, each planet is approximately a two-body problem in a central gravitational field. Bound Newtonian two-body orbits are ellipses; the numerical solver should remain close to those trajectories.

Exercise. Look at `orbits_keplerian.png`. The outer planets set the scale of the figure, so the inner Solar System appears crowded near the centre. What can you nevertheless infer about the shapes and orientations of the planetary orbits? Which properties are difficult to judge from this single 3-D view?

The distance plot `milankovitch_keplerian.png` shows the distance from each planet to the Sun against time. For an eccentric orbit, the minimum distance is the *perihelion* and the maximum distance is the *aphelion*.

Exercise. Look at the Pluto panel. Estimate Pluto's orbital period directly from the time between successive repeating features. Compare your estimate with Table 2.

Exercise. Earth's orbit is much less eccentric than Pluto's. How is this reflected in the amplitude of the Sun–planet distance variation?

6.2 Fourier analysis: finding periodic behaviour

A time series can be represented as a sum of sinusoidal components. A Fourier transform tells us how strongly different frequencies are represented in the signal. The supplied analysis script plots spectral amplitude against *time period* rather than frequency, so a peak at 12 years corresponds to a frequency of about $1/12$ yr^{-1}.

Run the Fourier analysis of the current Sun-only output:

```
python3 python/plot_solar_system03a.py
```

It writes `/tmp/$USER/fourier.png`. Download it using a distinct local filename:

```
get /tmp/YOUR_USERNAME/fourier.png fourier_keplerian.png
```

Exercise. Identify the dominant peak for as many planets as you can. How do the peak periods compare with Table 2? Why are the peaks for the outer planets harder to estimate precisely than those for the inner planets?

Now examine Earth's spectrum in more detail. Open the script:

```
nano -l python/plot_solar_system03a.py
```

Search for `xlim`. Change the plotting limit so that the active line reads:

```
plt.xlim((0.1, 1.2))
```

Save, rerun the script and download the new figure as `fourier_earth_zoom.png`.

Exercise. Besides the main orbital-period peak, identify several smaller peaks. Convert each period T to frequency $f = 1/T$. Divide the frequencies by the smallest frequency in the group. Are the ratios close to whole numbers?

Exercise. Why can a periodic signal contain harmonics as well as its fundamental frequency? Think about whether the planet–Sun distance varies as a perfect sine wave during an eccentric orbit.

6.3 Mutual planetary interactions

The default case deliberately suppresses planet–planet perturbations. We will now enable every body as a Newtonian gravity source. Open `namelist.in` and replace the active `interact` assignment with:

```
interact(1:10)=.true.,.true.,.true.,.true.,.true.,
               .true.,.true.,.true.,.true.,.true./
```

Leave `general_relativity=.false.` for this experiment.

Run the model again:

```
./run.sh
```

The new calculation overwrites `/tmp/$USER/output.nc`; this is why you downloaded the figures from the Sun-only run before changing the configuration.

Mutual planetary perturbations are one ingredient in the long-term variations of Earth's orbit that contribute to astronomical climate forcing. This model illustrates orbital perturbations; it is not by itself a complete model of Milankovitch cycles because it does not model all of the relevant rotational and orbital degrees of freedom.

Use the same Fourier script as before to look for an interaction timescale. Open it again and set:

```
plt.xlim((10, 30))
```

Run the script again and download the result as `fourier_saturn_interacting.png`.

Exercise. Look near Saturn's part of the spectrum. Can you identify a feature near 20 years that was not the main Saturn orbital period? This is associated with the Jupiter–Saturn synodic period: the time between successive conjunctions.

For orbital periods T_J and T_S, the synodic period T_{syn} satisfies

$$\frac{1}{T_{\mathrm{syn}}} = \left| \frac{1}{T_J} - \frac{1}{T_S} \right|. \tag{6}$$

Exercise. Using $T_J \approx 11.9$ yr and $T_S \approx 29.5$ yr, calculate T_{syn}. Compare your value with the feature in the Fourier plot. Why does an interaction between two periodic motions introduce a new timescale?

6.4 The Sun's wobble and exoplanets

A planet and its star both orbit their common centre of mass (barycentre). For the Solar System, Jupiter and the other giant planets cause the Sun to execute a small reflex motion. The same principle is used to detect exoplanets: a planet can be inferred from periodic motion of its host star, even when the planet itself is difficult to image directly.

For this exercise use the supplied Sun-wobble analysis script:

```
python3 python/plot_solar_system03b.py
```

It calculates the Solar System barycentre from the model output and analyses its displacement relative to the Sun. In a heliocentric output this has the same magnitude as the Sun's displacement relative to the barycentre. The script writes `/tmp/$USER/fourier_sun.png`. Download it as:

```
get /tmp/YOUR_USERNAME/fourier_sun.png
```

Exercise. Which periodicities have the strongest signal in `fourier_sun.png`? Compare them with the planetary orbital periods in Table 2. Which planet or planets have the largest influence on the Sun's reflex motion?

Exercise. For one planet with $m_p \ll M_\odot$, the characteristic stellar reflex distance scales approximately as

$$r_\star \sim \frac{m_p}{M_\odot} a_p,$$

where a_p is the planet's orbital size. Use this relation qualitatively to explain why the giant planets dominate the stellar wobble signal.

When you have finished, restore the supplied text files:

```
git reset --hard
```

7 What you should take away

The Solar System model is a concrete example of a large coupled ODE problem. Newton's law of gravitation supplies the accelerations, an ODE solver advances the positions and velocities, and the resulting time series can be analysed in both the time and frequency domains.

The practical also shows why model configuration matters. A Sun-only calculation produces idealised two-body motion; enabling mutual interactions introduces perturbations and combination timescales; barycentric analysis reveals the Sun's reflex motion about the system centre of mass. Fourier analysis provides a complementary way to identify periodic behaviour that may be difficult to isolate directly from a long time series.

The workflow should now be familiar: **obtain code → inspect configuration → compile → run → analyse → retrieve output → interpret the result**.